Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think

Sihyun Yu[1] Sangkyung Kwak[1] Huiwon Jang[1] Jongheon Jeong[2]
Jonathan Huang[3] Jinwoo Shin[1*] Saining Xie[4*]

[1]KAIST [2]Korea University [3]Scaled Foundations [4]New York University
[*]Equal Advising

ICLR 2025 (Oral)

Outline

1. Introduction & Motivation
2. Key Observations
3. Method: REPA
4. Bridging the Representation Gap
5. Experimental Results
6. Conclusion
7. Ablation Studies

Motivation

Diffusion Transformers (DiT, SiT) are the new paradigm for image generation:

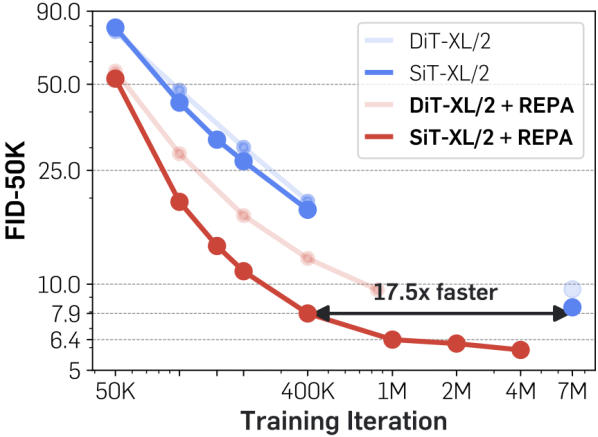
- State-of-the-art on ImageNet generation
- Scale well with model size and compute

Key challenge:

- Training is extremely expensive (SiT-XL/2: **7M steps**)
- Bottleneck: Learning good internal representations

This paper's insight:

- Use **pretrained visual encoders** to guide representation learning
- Simple regularization $\Rightarrow$ **17.5$\times$ faster** training



Background: Flow Matching & Diffusion Transformers

Flow Matching / Interpolant Framework:
Given clean data x_0 and noise $\epsilon \sim \mathcal{N}(0, I)$, construct noisy input:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \tag{1}$$

Linear path: $\alpha_t = 1 - t, \quad \sigma_t = t, \qquad t \in [0, 1]$

Velocity (v-prediction) target:

$$v_t = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon \tag{2}$$

For linear path: $v_t = -x_0 + \epsilon$

Diffusion Transformers:

- **DiT** (Peebles & Xie, 2023): Transformer backbone replaces U-Net
- **SiT** (Ma et al., 2024): DiT + Scalable Interpolant framework
- Patch embedding $\rightarrow$ Transformer blocks with adaLN-Zero $\rightarrow$ Unpatchify

Background: Self-Supervised Visual Encoders

Contrastive / Self-distillation:

- **DINOv2** – Self-distillation with ViT
- **MoCo v3** – Momentum contrastive
- **CLIP** – Vision-language contrastive

Masked prediction:

- **MAE** – Masked autoencoder
- **I-JEPA** – Joint-embedding predictive

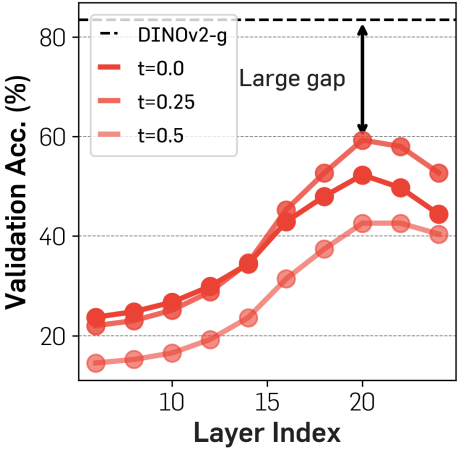
Key question: *Can we leverage these high-quality representations to help train diffusion models?*

Observation 1: Diffusion Models Learn Representations

Layer-wise linear probing on a pretrained SiT-XL/2 (7M steps):

- SiT *does* learn meaningful discriminative features
- But quality significantly lags behind DINOv2
- Middle-to-later layers show best probing accuracy

Takeaway: There is a large representation gap between diffusion models and SSL encoders.

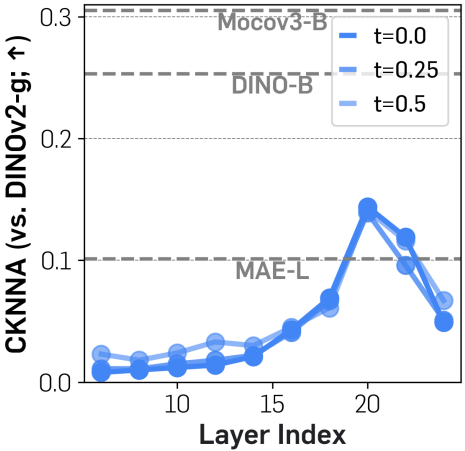


Observation 2: Weak Alignment with SSL Representations

Centered Kernel Alignment (CKA) between SiT and DINOv2:

- Measures similarity between two sets of representations
- Alignment is weak across all layers
- Early layers: almost no alignment
- Later layers: moderate but still far from perfect

Despite learning useful features, SiT representations are poorly aligned with DINOv2.

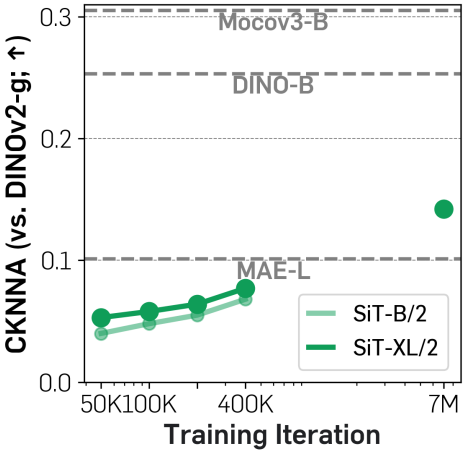


Observation 3: Alignment Improves with Longer Training

CKA alignment between SiT and DINOv2 at different training stages:

- Alignment **consistently improves** with more training steps
- Larger models align faster
- This suggests: *alignment is a natural tendency, just very slow*

Hypothesis: If we explicitly encourage this alignment, training will converge much faster!



Summary of Observations

Obs 1: Diffusion models learn discriminative representations, but they are inferior to SSL encoders (DINOv2)

↓

Obs 2: The alignment between diffusion model and DINOv2 representations is weak

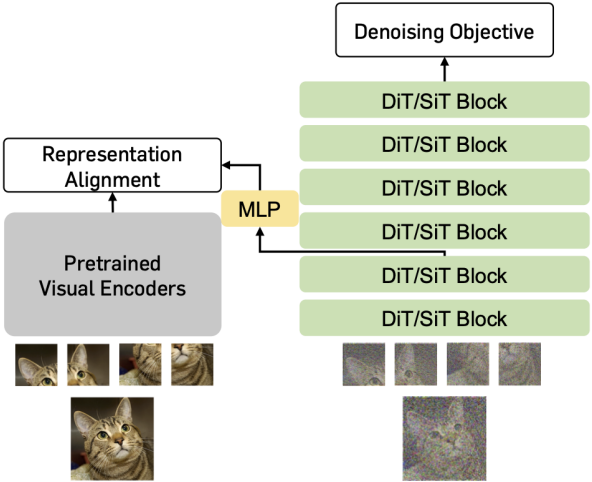
↓

Obs 3: This alignment naturally improves with longer training and larger models

↓

Idea: *Explicitly* align diffusion model hidden states with SSL representations ⇒ **REPA**

REPA: REPresentation Alignment



Core idea:

Align the hidden representation of the denoising network (at an intermediate layer l) with the clean image representation from a frozen pretrained encoder.

Key components:

- Frozen SSL encoder (DINOv2)
- Projector MLP on hidden states
- Cosine similarity alignment loss
- No overhead at inference

Flow Matching Formulation

Stochastic interpolant:

$$x_t = \alpha_t\, x_0 + \sigma_t\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad t \in [0, 1] \tag{3}$$

Path types:

- **Linear:** $\alpha_t = 1 - t, \quad \sigma_t = t$
- **Cosine:** $\alpha_t = \cos\left(\frac{\pi t}{2}\right), \quad \sigma_t = \sin\left(\frac{\pi t}{2}\right)$

Velocity field (v-prediction):

$$v_t = \dot{\alpha}_t\, x_0 + \dot{\sigma}_t\, \epsilon \tag{4}$$

The model $v_\theta(x_t, t)$ is trained to predict this velocity.

Denoising loss:

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}_{t, x_0, \epsilon}\left[\| v_\theta(x_t, t) - v_t \|^2\right] \tag{5}$$

REPA Alignment Loss

Setup: Given a frozen pretrained encoder f_{enc} (e.g., DINOv2):

1. Extract **clean image features**: $z = f_{\text{enc}}(x_0) \in \mathbb{R}^{N \times d_z}$ (patch tokens)
2. Extract **noisy hidden states** at layer l: $h^{(l)} = \text{DiT-Block}_l(x_t, t, y)$
3. Project hidden states: $\tilde{z} = f_{\text{proj}}(h^{(l)}) \in \mathbb{R}^{N \times d_z}$

Alignment loss (negative cosine similarity):

$$\mathcal{L}_{\text{align}} = -\mathbb{E}_{t, x_0, \epsilon} \left[\frac{1}{N} \sum_{j=1}^{N} \left\langle \frac{\tilde{z}_j}{\|\tilde{z}_j\|}, \frac{z_j}{\|z_j\|} \right\rangle \right] \tag{6}$$

where N is the number of patch tokens, and the sum is over **spatial positions**.

Intuition: Force the diffusion model's intermediate representation to "see" what DINOv2 sees in the clean image, even from noisy input.

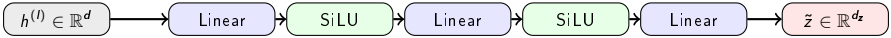
Total Training Objective

Combined loss:

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{denoise}} + \lambda \cdot \mathcal{L}_{\text{align}}} \tag{7}$$

where λ controls the alignment strength (default: $\lambda = 0.5$).

Projector MLP f_{proj}: maps hidden states to encoder space



Dimensions: $d \to 2048 \to 2048 \to d_z$ (projector_dim = 2048)

Design Choices

Which layer to align?

- Alignment applied at layer l = `encoder_depth`
- Default: $l = 8$ for SiT-XL/2 (28 total blocks)
- **Early layers** $\Rightarrow$ aligns low-level + semantic features

Which encoder?

- **DINOv2** (ViT-B, ViT-L, ViT-g) – best overall
- Also works with: CLIP, MoCo v3, I-JEPA, MAE, DINOv1

Key design principle:

Align **early layers** with SSL encoder
$\Rightarrow$ Later layers are freed to focus on **high-frequency generation details**

Other details:

- Projector is *only used during training* (discarded at inference)
- Encoder is *frozen* – no gradient flows back
- $\lambda = 0.5$, AdamW optimizer, lr $= 10^{-4}$, batch size $= 256$

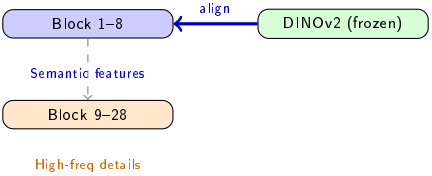
How REPA Works: Layer Specialization

Without REPA (Vanilla):

- All layers must jointly learn both *semantic understanding* and *fine-grained generation*
- Slow convergence – early layers struggle

With REPA:

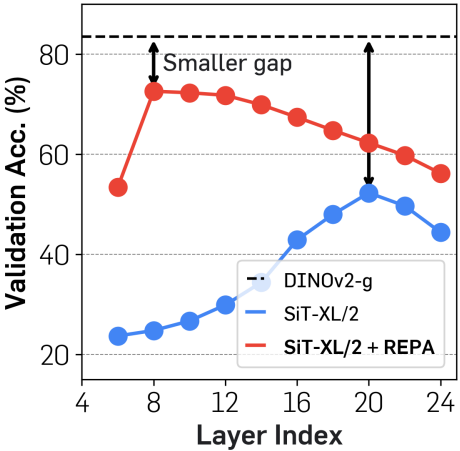
- **Early layers** ($\leq l$): Aligned with SSL encoder $\Rightarrow$ semantics "injected"
- **Later layers** ($> l$): Free for high-freq details



REPA Bridges the Representation Gap: Linear Evaluation

Linear probing accuracy per layer:

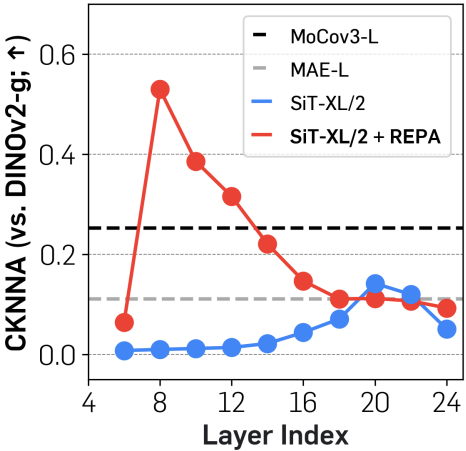
- REPA dramatically improves representation quality at **early layers**
- Representations become comparable to DINOv2 in quality
- This confirms: alignment is *successfully transferred*



REPA Bridges the Representation Gap: CKA Alignment

CKA alignment with DINOv2:

- REPA achieves **much stronger** alignment, especially at early layers
- Alignment at layer 8 (the target) is very high
- Later layers remain *different* – they specialize in generation

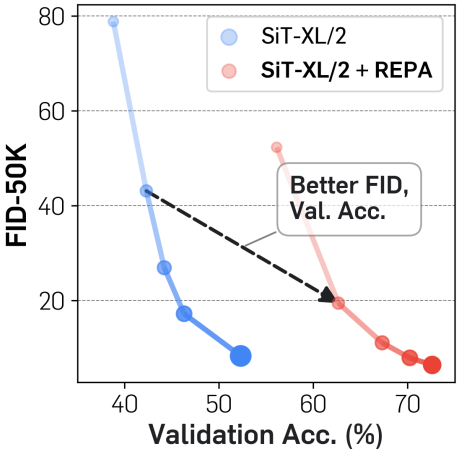


Layer Specialization: Representation Slope

Representation slope measures how much each layer changes the representation:

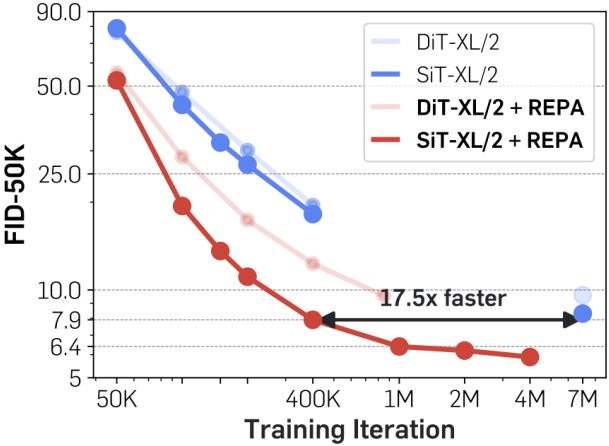
- With REPA: a **clear transition** at layer l
- Layers $\leq l$: smooth, aligned representations
- Layers $> l$: sharp changes for generation-specific processing

Conclusion: REPA creates a natural **division of labor** between semantic encoding and generative decoding.

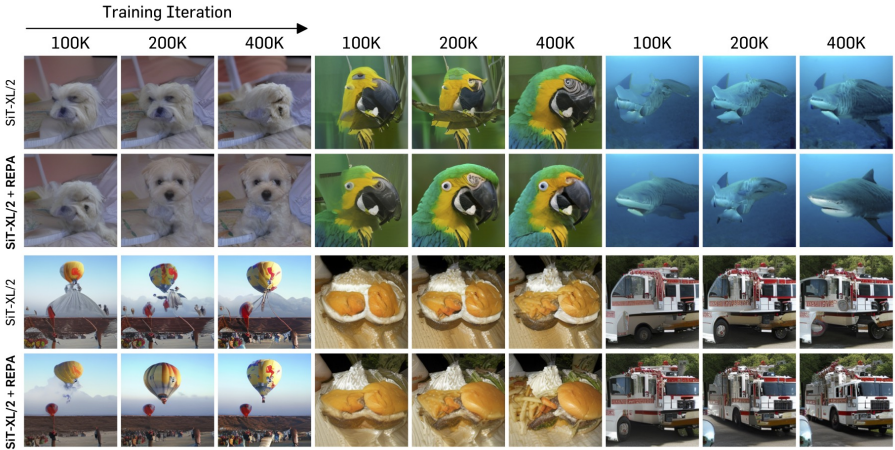


Result 1: 17.5× Faster Training

- REPA matches vanilla SiT-XL/2 (7M steps) in <**400K steps**
- **17.5× speedup** in training convergence
- Consistent improvement across FID, IS, and linear probing metrics



Result 2: Qualitative Progression During Training



Top: Vanilla SiT-XL/2 **Bottom**: SiT-XL/2 + REPA

Same noise, same sampler, same steps. REPA produces recognizable images much earlier.

Result 3: Generation Quality without CFG

- Without CFG, REPA achieves **FID = 7.9** at only 400K steps
- Outperforms vanilla SiT-XL/2 trained for **7M steps** (FID = 9.9)
- Also improves Inception Score and linear probing accuracy

Model	#Params	Iter.	FID↓
DiT-L/2	458M	400K	23.3
+ REPA (ours)	458M	**400K**	**15.6**
DiT-XL/2	675M	400K	19.5
+ REPA (ours)	675M	**400K**	**12.3**
DiT-XL/2	675M	**7M**	9.6
+ REPA (ours)	675M	**850K**	**9.6**
SiT-B/2	130M	400K	33.0
+ REPA (ours)	130M	**400K**	**24.4**
SiT-L/2	458M	400K	18.8
+ REPA (ours)	458M	**400K**	**9.7**
+ REPA (ours)	458M	**700K**	**8.4**
SiT-XL/2	675M	7M	8.3
+ REPA (ours)	675M	**400K**	**7.9**
+ REPA (ours)	675M	**1M**	**6.4**
+ REPA (ours)	675M	**4M**	**5.9**

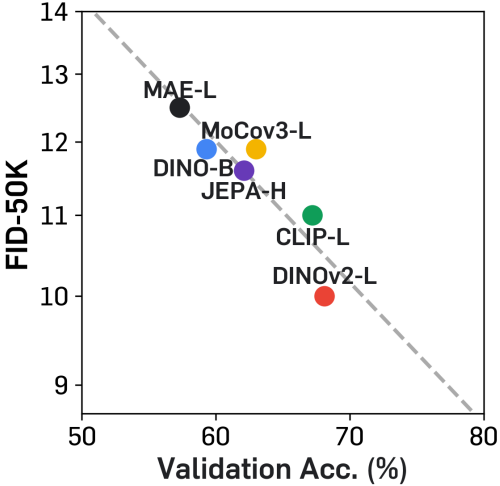
Result 4: State-of-the-Art with CFG

- With CFG + guidance interval, REPA achieves **FID = 1.42** (state-of-the-art)
- Outperforms all previous diffusion models with **7× fewer epochs**
- Works with both SDE and ODE sampling

Model	Epochs	FID↓	sFID↓	IS↑	Pre.↑	Rec.↑
Pixel diffusion						
ADM-U	400	3.94	6.14	186.7	0.82	0.52
VDM++	560	2.40	-	225.3	-	-
Simple diffusion	800	2.77	-	211.8	-	-
CDM	2160	4.88	-	158.7	-	-
Latent diffusion, U-Net						
LDM-4	200	3.60	-	247.7	0.87	0.48
Latent diffusion, Transformer + U-Net hybrid						
U-ViT-H/2	240	2.29	5.68	263.9	0.82	0.57
DiffiT*	-	1.73	-	276.5	0.80	0.62
MDTv2-XL/2*	1080	1.58	4.52	314.7	0.79	0.65
Latent diffusion, Transformer						
MaskDiT	1600	2.28	5.67	276.6	0.80	0.61
SD-DiT	480	3.23	-	-	-	-
DiT-XL/2	1400	2.27	4.60	278.2	**0.83**	0.57
SiT-XL/2	1400	2.06	4.50	270.3	0.82	0.59
+ REPA (ours)	200	1.96	**4.49**	264.0	0.82	0.60
+ REPA (ours)	800	1.80	4.50	284.0	0.81	0.61
+ REPA (ours)*	**800**	**1.42**	4.70	**305.7**	0.80	**0.65**

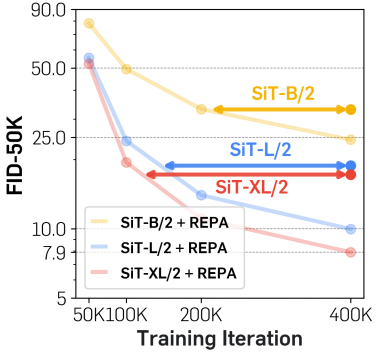
Result 5: Scalability Across Encoders

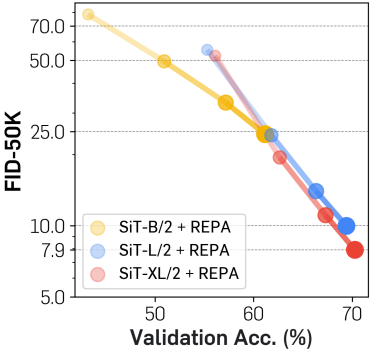
- Better encoder ⇒ better generation **and** representation
- **DINOv2** family performs best overall
- Even weaker encoders (MAE, DINOv1) improve over vanilla



Result 6: Scaling Laws

- REPA provides **greater speedup for larger models**
- **Log-linear correlation** between linear probing accuracy and FID
- Improving representations directly translates to better generation





Qualitative Samples (SiT-XL/2 + REPA)

Key Takeaways

1. **Representation quality is the bottleneck** in training diffusion transformers
2. **REPA** is a simple regularization: align diffusion model hidden states with pretrained SSL encoders using cosine similarity
3. Results are **striking**:
   - **17.5**$\times$ training speedup
   - State-of-the-art **FID = 1.42** on ImageNet 256$\times$256
   - Works with diverse encoders (DINOv2, CLIP, I-JEPA, MAE, ...)
4. REPA creates **layer specialization**: early layers for semantics, later layers for generation details
5. **Practical takeaway:** Training powerful diffusion transformers is now *much easier and cheaper*

Limitations & Future Work

Limitations:

- Requires access to a pretrained visual encoder (additional dependency)
- Currently demonstrated primarily on ImageNet – scaling to diverse domains needs study
- The projector MLP adds a small overhead during training (discarded at inference)

Future directions:

- Apply REPA to **text-to-image** generation at scale (initial MS-COCO results shown)
- Explore alignment with **multi-modal encoders** (e.g., CLIP for text-guided alignment)
- Combine with **progressive training** or **distillation** for even faster convergence
- Investigate **adaptive alignment depth** – dynamically choose layer l

Ablation: Alignment Depth l (Table 2)

SiT-L/2, 400K iterations, DINOv2-L encoder, NT-Xent loss, without CFG:

Depth l	FID↓	sFID↓	IS↑	Pre.↑	Rec.↑	Acc.↑
Vanilla	18.8	5.29	72.0	0.64	0.64	N/A
6	10.3	5.23	106.5	0.69	0.65	66.2
8	**10.0**	**5.09**	**106.6**	**0.68**	**0.65**	**68.1**
10	10.5	5.50	105.0	0.68	0.65	68.6
12	11.2	5.14	100.2	0.68	0.64	69.4
14	11.6	5.61	99.5	0.67	0.65	70.0
16	12.1	5.34	96.1	0.67	0.64	71.1

Key findings:

- Aligning the **first 8 layers** gives the best FID for SiT-L/2 (24 blocks total)
- Deeper alignment improves probing accuracy but *hurts* generation quality
- Hypothesis: later layers need freedom to focus on high-frequency details

Ablation: Alignment Coefficient λ (Table 5)

SiT-XL/2, 400K iterations, without CFG:

λ	**FID-50K** $\downarrow$	**IS** $\uparrow$
0.25	8.6	118.6
0.5	**7.9**	**122.6**
0.75	7.8	124.4
1.0	7.8	124.8

Key findings:

- $\lambda = 0.5$ chosen as default – good balance between denoising and alignment
- Results are **stable** across a wide range $\lambda \in [0.5, 1.0]$
- Even small $\lambda = 0.25$ yields significant improvement over vanilla

Ablation: Alignment Objective (Table 2)

SiT-L/2, DINOv2-L, depth=8, 400K iterations, without CFG:

Objective	FID↓	sFID↓	IS↑	Pre.↑	Acc.↑
NT-Xent	10.0	5.09	106.6	0.68	68.1
Cos. sim.	**9.9**	**5.34**	**111.9**	**0.68**	**68.2**

Key findings:

- NT-Xent has advantages at early training stages (50–100K iters)
- At 400K iters, **cosine similarity** achieves equivalent or better results
- Cos. sim. adopted as default for simplicity and efficiency

Appendix A: Diffusion-Based Models Used

Two main training frameworks used in the paper:

DiT (Peebles & Xie, 2023):

- Objective: improved DDPM (Nichol & Dhariwal, 2021)
- Predicts noise ϵ
- Scheduler: cosine noise
- Patch size: 2 ($256 \times 256 \rightarrow 256$ tokens)

SiT (Ma et al., 2024):

- Objective: linear stochastic interpolants
- Predicts velocity v
- Sampler: SDE Euler-Maruyama (NFE=250)
- Last SDE step = 0.04 (crucial)

Note: REPA applies to both frameworks; main results use **SiT** as the SDE sampler yields higher quality.

Appendix B: Diffusion Transformer Architecture

Model configurations (Table 1):

Config	Layers	Hidden	Heads
B/2	12	768	12
L/2	24	1024	16
XL/2	28	1152	16

- Patch size = 2 ($32\times32 \rightarrow 256$ tokens)
- VAE latent: $32\times32\times4$ (SD-VAE)
- Class conditioning via adaLN-Zero

DiT Block:

- adaLN-Zero: scale/shift from timestep + class embedding
- Self-Attention (multi-head)
- Feed-Forward (MLP)
- Skip connection + Layer Norm

REPA projection:

- 3-layer MLP (SiLU) from $h^{(l)}$
- Attached to first l layers only
- Discarded at inference

Appendix C: CKNNA Metric Definition

CKNNA = Centered Kernel Alignment with k-Nearest Neighbors:

Original CKA (Kornblith et al., 2019):

$$\mathrm{CKA}(K, L) = \frac{\mathrm{HSIC}(K, L)}{\sqrt{\mathrm{HSIC}(K, K) \cdot \mathrm{HSIC}(L, L)}}$$

CKNNA (Huh et al., 2024) replaces HSIC with $\mathrm{Align}(K, L)$ using:

$$\alpha(i, j;\, k) = 1\big[i \neq j \;\wedge\; \phi_j \in \mathrm{knn}(\phi_i;\, k) \;\wedge\; \psi_j \in \mathrm{knn}(\psi_i;\, k)\big]$$

Experimental details:

- $k = 10$ nearest neighbors
- Random sample of **10,000** ImageNet validation images
- Linear probing: 90 epochs, batch=16384, Adam, cosine decay, LR= 10^{-3}

CKNNA is more sensitive than CKA for measuring local (neighborhood-level) alignment between feature spaces.

Appendix D: Training Hyperparameters

General hyperparameters:

Parameter	Value
Optimizer	AdamW
Learning rate	10^{-4} (constant)
(β_1, β_2)	(0.9, 0.999)
Weight decay	0
Batch size	256
Mixed precision	FP16
Grad clipping	norm ≤ 1.0
Sampler (eval)	SDE EM, NFE=250
Last SDE step	0.04

REPA-specific (default):

Parameter	Value
λ	0.5
Encoder depth /	8
Encoder	DINOv2-B
Similarity	Cos. sim.
Projector dim	2048
Iterations	400K / 4M

Compute:

- 8× NVIDIA H100 80GB
- ≈5.4 steps/s (batch=256)
- VAE latents pre-computed

Appendix E+F: Evaluation & Baselines

Appendix E – Metrics:

- **FID**: Fréchet Inception Distance (50K samples)
- **sFID**: spatial FID (patch features)
- **IS**: Inception Score
- **Pre./Rec.**: precision & recall (Kynkäänniemi et al., 2019)
- **Acc.**: linear probing on ImageNet val
- **CKNNA**: alignment score (see App. C)

Appendix F – Baselines:

- **Pixel diffusion**: ADM, VDM++, Simple diffusion, CDM
- **Latent U-Net**: LDM
- **Transformer+U-Net hybrid**: U-ViT-H/2, DiffiT, MDTv2-XL/2
- **Pure latent DiT**: MaskDiT, SD-DiT, DiT-XL/2, SiT-XL/2

All comparisons use the same VAE and evaluation protocol.

Appendix G: Detailed Results – Without CFG (Table 8)

Model-scale comparison without CFG:

Model	Iter.	FID↓	sFID↓	IS↑	Pre.↑	Rec.↑	Acc.↑
SiT-B/2 vanilla	400K	33.0	6.46	43.7	0.53	0.63	—
SiT-B/2+REPA	400K	24.4	6.40	59.9	0.59	0.65	61.2
SiT-L/2 vanilla	400K	18.8	5.29	72.0	0.64	0.64	—
SiT-L/2+REPA	400K	10.0	5.20	109.2	0.69	0.65	69.4
SiT-XL/2 vanilla	7M	8.3	6.32	131.7	0.68	0.67	—
SiT-XL/2+REPA	400K	7.9	5.06	122.6	0.70	0.65	70.3
SiT-XL/2+REPA	**4M**	**5.9**	5.73	**157.8**	**0.70**	**0.69**	**74.6**

Note: REPA+SiT-XL/2 at 400K iterations already surpasses vanilla SiT-XL/2 trained for 7M iterations.

Appendix G: Detailed Results – With CFG (Tables 9–10)

SiT-XL/2+REPA, 4M iterations, with guidance interval (Kynkäänniemi et al., 2024):

Best result (Table 9): interval$= [0, 0.7]$, $w = 1.80$:

Interval	FID↓	sFID↓	IS↑	Pre.↑	Rec.↑
$[0, 0.7]$, $w = 1.80$	**1.42**	4.70	305.7	0.80	0.64

Guidance interval:

- Apply CFG only within the noise range $[t_{\text{low}}, t_{\text{high}}]$
- Reduces artifacts from over-guidance at low noise levels
- SDE sampler + interval $[0, 0.7]$ + $w = 1.80 \Rightarrow$ **FID = 1.42** (state-of-the-art)

Appendix J: ImageNet 512×512

Setup: SD-VAE encodes to 64×64×4 latent (1024 tokens); DINOv2 input resized to 448×448.

Table 11: System-level comparison on ImageNet 512×512 (CFG w=1.35):

Model	Epochs	FID↓	sFID↓	IS↑	Pre.↑
MaskDiT	800	2.50	5.10	256.3	0.83
DiT-XL/2	600	3.04	5.02	240.8	0.84
SiT-XL/2	600	2.62	4.18	252.2	0.84
+REPA	80	2.44	4.21	247.3	0.84
+REPA	100	2.32	4.16	255.7	0.84
+REPA	**200**	**2.08**	4.19	**274.6**	0.83

Key finding: REPA at **200 epochs** surpasses vanilla SiT-XL/2 at **600 epochs** – **>3× fewer training iterations**.

Appendix K: Text-to-Image Generation (MS-COCO)

Setup:

- Model: **MMDiT** (Multi-Modal DiT)
- Visual encoder: DINOv2-ViT-B (frozen)
- Text encoder: CLIP (frozen)
- Dataset: MS-COCO (train/val)
- 150K iterations, batch=256
- Hidden=768, depth=24

Key results:

- ODE: FID 6.05 $\rightarrow$ 4.73
- SDE: FID 5.30 $\rightarrow$ 4.14
- Effective **independently** of text conditioning

Table 12: MS-COCO FID (CFG w=2.0):

Method	**Type**	**FID**$\downarrow$
U-ViT-S/2	Diffusion	5.95
U-ViT-S/2 Deep	Diffusion	5.48
MMDiT (ODE)	Diffusion	6.05
MMDiT+REPA (ODE)	Diffusion	**4.73**
MMDiT (SDE)	Diffusion	5.30
MMDiT+REPA (SDE)	Diffusion	**4.14**

REPA is architecture-agnostic and task-agnostic.

Appendix L: Feature Map Visualization (PCA)

PCA visualization similar to DINOv2 – analyzing features by layer and timestep:

With REPA (SiT-XL/2+REPA):

- Early layers (1–6): **coarse**, high-level semantic features
- Later layers (14–28): **fine-grained**, local detail features
- Clear coarse-to-fine pattern at all timesteps t

Without REPA (Vanilla):

- Features are **noisy** at large t ($t = 0.7, 0.9$)
- No clear hierarchical structure
- Weak representation alignment

Figure 38: layers $\{1, 2, 3, 4, 5, 6, 7, 14, 21, 28\}$ at $t \in \{0.5, 0.7, 0.9\}$ – REPA learns SSL-like coarse-to-fine features; vanilla shows noisy, unstructured features.

Appendix M: Limitations & Future Work

Current limitations of REPA:

- Requires access to a pretrained SSL encoder (additional dependency)
- Primarily demonstrated on ImageNet – scaling to diverse domains needs study
- Projector MLP adds a small training overhead (discarded at inference)

Four future research directions (Appendix M):

1. **Alignment depth analysis**: theoretical understanding of why layer 8 is optimal
2. **Broader data types**: pixel-level, video, large-scale T2I at scale
3. **Theoretical analysis**: relationship between instance discrimination and denoising
4. **Time-varying REPA**: weight function $\lambda(t)$ varying with the noise schedule

Thank for your listening!

Paper: https://arxiv.org/abs/2410.06940
Code: https://github.com/sihyun-yu/REPA
Project: https://sihyun.me/REPA/